MODULE 2 – SCÉNARIO 6

Devoir d'analyse – Analyse des SMS d'urgence

Logique de programmation, NumPy et simulation • Travail individuel

Informations	
Nom et prénom	Arismé Djephane Asline Gabriel
Date	27 octobre 2026
Module	2
Scénario	6 – Analyse des SMS d'urgence [MISSION EXPRESS]
Date de remise	Mercredi 28 octobre 2026

1. Contexte du devoir

Le 3 septembre 2026 au matin, la coordination de terrain reçoit des SMS de relais communautaires du Sud : un besoin en eau à Torbeck, une route bloquée à Chantal, des abris détruits à Maniche. Chaque message suit en principe le format « LIEU|THÈME|STATUT|NOMBRE|HORODATAGE », mais il est tapé vite, sur un vieux téléphone et parfois sous la pluie. Certains SMS sont mal formés, et d'autres sont des renvois du même message.

Vous êtes data scientist. La coordinatrice de terrain a besoin de savoir combien de personnes sont réellement concernées par thème (eau, route, abri, nourriture…). Si l'on additionne simplement tous les SMS, les renvois gonflent les chiffres. Vous devez donc comparer, pour chaque thème, le total obtenu en additionnant les SMS et le total obtenu après dédoublonnage par événement, puis rédiger un rapport destiné au coordinateur ou à la coordinatrice du projet.

Données : `S06_sms_urgence.txt` (données synthétiques).

2. Méthodologie obligatoire

1. Ouvrir le notebook du Module 2 et aller au Scénario 6 : « Analyse des SMS d'urgence ».
2. Téléverser le fichier `S06_sms_urgence.txt` et exécuter le code de départ (partie 3). Sans Colab, suivre la partie 4.
3. Vérifier les deux contrôles : SMS lus = valides + rejetés, et chaque SMS valide rattaché à un événement.
4. Écrire vous-même le code qui produit le tableau comparatif par thème (parties 5 et 6).
5. Vérifier vos résultats, puis rédiger le rapport destiné au coordinateur ou à la coordinatrice du projet (partie 7).
6. Redémarrer le noyau et exécuter toutes les cellules dans l'ordre avant la remise.

3. Code de départ (déjà présent dans le notebook)

Ce code lit les SMS, sépare les valides des rejetés, puis regroupe les renvois en événements distincts. Il produit deux objets dont vous aurez besoin : `valides` (liste des SMS valides) et `evenements` (tableau des événements dédoublonnés).

```
# --- Préparation (déjà présente dans le notebook) ---
from google.colab import files
import pandas as pd
from IPython.display import display
from datetime import datetime, timedelta

# Téléverser le fichier S06_sms_urgence.txt
uploaded = files.upload()
nom = list(uploaded.keys())[0]

# Lire le fichier texte et le découper en lignes
lignes = uploaded[nom].decode("utf-8").strip().splitlines()
entete, messages = lignes[0], lignes[1:]          # en-tête + SMS
print(f"{len(messages)} SMS lus. En-tête : {entete}")

# Transformer un SMS brut en dictionnaire, ou lever une erreur
def lire_sms(texte):
    """Découpe 'LIEU|THEME|STATUT|NOMBRE|HORODATAGE'.
    Lève ValueError si le SMS est invalide."""
    morceaux = [m.strip() for m in texte.split("|")]
    if len(morceaux) != 5:
        raise ValueError(f"{len(morceaux)} éléments au lieu de 5")
    lieu, theme, statut, nombre, horodatage = morceaux
    if not nombre.isdigit():
        raise ValueError("Le nombre doit contenir uniquement des chiffres")
    return {
        "lieu": lieu.title(),
        "theme": theme.upper(),
        "statut": statut.upper(),
        "nombre": int(nombre),
        "horodatage": datetime.strptime(horodatage, "%Y-%m-%d %H:%M"),
    }

# Séparer les SMS valides et les SMS rejetés
valides, rejets = [], []
for numero, texte in enumerate(messages, start=1):
    try:
        valides.append(lire_sms(texte))
    except ValueError as erreur:
        rejets.append({"ligne": numero, "sms": texte, "motif": str(erreur)})

# Contrôle 1 : SMS lus = valides + rejetés
assert len(messages) == len(valides) + len(rejets)
print(f"Validés : {len(valides)} | Rejetés : {len(rejets)}")
display(pd.DataFrame(rejets))
display(pd.DataFrame(valides))

# Dédoublonnage : même lieu, thème, statut, nombre dans une fenêtre de 3 h
```

```
FENETRE = timedelta(hours=3)
CLES = ["lieu", "theme", "statut", "nombre"]
evenements = []
for sms in sorted(valides, key=lambda s: s["horodatage"]):
    for ev in evenements:
        if all(ev[c] == sms[c] for c in CLES) and \
           sms["horodatage"] - ev["premier_sms"] <= FENETRE:
            ev["nb_sms"] += 1          # renvoi du même événement
            break
    else:
        evenements.append({**{c: sms[c] for c in CLES},
                           "premier_sms": sms["horodatage"], "nb_sms": 1})

evenements = pd.DataFrame(evenements)

# Contrôle 2 : chaque SMS valide est rattaché à un événement
assert evenements["nb_sms"].sum() == len(valides)
display(evenements)
```

4. Travailler sans Colab (VS Code ou PyCharm)

Si vous ne pouvez pas utiliser Google Colab, vous pouvez faire le devoir sur votre ordinateur avec VS Code ou PyCharm. Deux options sont possibles.

Option A – garder le notebook (recommandé)

1. Dans Colab : **Fichier → Télécharger → Télécharger le fichier .ipynb**.
2. Installer Python 3, puis dans un terminal : `pip install pandas numpy jupyter`.
3. VS Code : installer les extensions **Python** et **Jupyter**, puis ouvrir le fichier .ipynb. PyCharm : ouvrir directement le fichier .ipynb.
4. Placer `S06_sms_urgence.txt` dans le même dossier que le notebook, puis appliquer les modifications ci-dessous.

Option B – convertir le notebook en fichier .py

1. Dans Colab : **Fichier → Télécharger → Télécharger le fichier .py**. Ou, si Jupyter est installé, dans un terminal : `jupyter nbconvert --to script nom_du_notebook.ipynb`.
2. Installer les bibliothèques : `pip install pandas numpy`.
3. Placer `S06_sms_urgence.txt` dans le même dossier que le fichier .py.
4. Appliquer les modifications ci-dessous, puis exécuter le script : `python nom_du_fichier.py` (ou bouton ▶ Run dans VS Code / PyCharm).

Modifications obligatoires hors de Colab :

- Supprimer toutes les lignes `from google.colab import files` et `uploaded = files.upload()` : elles ne fonctionnent que dans Colab.
- Remplacer la lecture du fichier téléversé par une lecture directe du fichier :

```
# Lire le fichier placé dans le même dossier que le notebook / le script
with open("S06_sms_urgence.txt", encoding="utf-8") as f:
    lignes = f.read().strip().splitlines()
```

- Dans un fichier .py (option B) : remplacer `display(...)` par `print(...)` pour afficher les tableaux. Dans un notebook (option A), `display` fonctionne normalement.

5. Tableau comparatif par thème (10 points)

Consigne : produisez un tableau qui compare, pour chaque thème, le nombre de personnes concernées selon deux méthodes. Le tableau contient une colonne avec tous les thèmes et deux colonnes de résultats :

Colonne	Contenu
Thème	Tous les thèmes présents dans les SMS valides (une ligne par thème)
Personnes_si_on_additionne_les_sms	Somme du champ « nombre » sur tous les SMS valides du thème, renvois compris
Personnes_par_évènement	Somme du champ « nombre » sur les événements dédoublonnés du thème

Recopiez votre tableau final ci-dessous, ou insérez une capture d'écran de sa sortie.

Thème	Personnes_si_on_additionne_les_sms	Personnes_par_évènement
ABRI	8	8
EAU	62	38
NOURRITURE	5	5
ROUTE	6	3

6. Code qui génère le tableau (30 points)

Consigne : à la suite du code de départ, écrivez vous-même le code qui produit le tableau de la partie 5, en utilisant **pandas** :

1. Transformer la liste `valides` en DataFrame avec `pd.DataFrame(...)` ;
2. Sur ce DataFrame, regrouper les SMS par thème avec `groupby("theme")` et faire la somme de la colonne `"nombre"` : c'est le total en additionnant les SMS ;
3. Faire le même regroupement par thème sur le DataFrame `evenements` : c'est le total par événement ;
4. Réunir les deux résultats dans un nouveau `pd.DataFrame` dont les colonnes s'appellent `Personnes_si_on_additionne_les_sms` et `Personnes_par_évènement` (les thèmes forment l'index) ;
5. Afficher le tableau final avec `display(...)`.

Commentez chaque ligne de votre code.

Votre code (ou capture d'écran de la cellule et de sa sortie) :

```python
 # Conversion de la liste valides en DataFrame
df_valides = pd.DataFrame(valides)

# Regroupement par thème et sommation de la colonne nombre (total brut)
total_sms_brut = df_valides.groupby("theme")["nombre"].sum()

# Regroupement similaire sur le DataFrame des événements (total par événement)
total_evenements = evenements.groupby("theme")["nombre"].sum()

# Fusion des deux résultats dans un nouveau DataFrame
tableau_comparatif = pd.DataFrame({
    "personnes_si_on_additionne_les_sms": total_sms_brut,
    "personnes_par_evenement": total_evenements
})

# Affichage du tableau comparatif final
print(tableau_comparatif)
```

theme	personnes_si_on_additionne_les_sms	personnes_par_evenement
ABRI	8	8
EAU	62	38
NOURRITURE	5	5
ROUTE	6	3

7. Rapport au coordinateur du projet (60 points)

Rédigez un rapport d'au moins **20 lignes**, accompagné des tableaux listés ci-dessous, destiné au coordinateur ou à la coordinatrice du projet. Le rapport présente vos constats et les recommandations qui en découlent. Il s'adresse à une personne qui n'a pas besoin de connaître le code Python : expliquez ce que les résultats signifient pour la décision, pas comment le programme fonctionne.

À : Coordinateur ou coordinatrice du projet

Objet : Analyse des SMS d'urgence du 3 septembre 2026 – constats et recommandations

Tableaux à intégrer dans le rapport :

Tableau	Contenu
Tableau 1	Bilan des contrôles : nombre de SMS lus, valides, rejetés et nombre d'événements distincts
Tableau 2	SMS rejetés, avec leur numéro de ligne et le motif du rejet
Tableau 3	Événements dédoublonnés : lieu, thème, statut, nombre de personnes, premier SMS, nombre de SMS reçus
Tableau 4	Tableau comparatif par thème (partie 5)

Chaque tableau doit avoir un titre et être commenté dans le texte. Les tableaux ne comptent pas dans les 20 lignes.

Le texte du rapport doit :

- présenter brièvement le contexte et l'objectif de l'analyse ;
- présenter des résultats chiffrés avec leur unité (nombre de personnes, nombre de SMS) ;
- expliquer pourquoi additionner tous les SMS surestime les besoins, et de combien ;
- identifier le ou les thèmes prioritaires et les lieux concernés ;
- discuter les SMS rejetés : quelles informations ont été perdues et quel risque cela représente ;
- mentionner au moins une limite de l'analyse ;
- proposer des recommandations fondées sur les constats (intervention, collecte des SMS, qualité des données) ;
- préciser que les données sont synthétiques.

Texte du rapport :

À : Coordinateur ou coordinatrice du projet

Objet : Analyse des messages d'urgence du 3 septembre 2026 – constats et recommandations

L'analyse présentée dans ce rapport porte sur les messages d'urgence reçus le 3 septembre 2026. L'objectif principal de cette étude est d'étudier les besoins de la population pour aider à la prise de décision rapide sur le terrain.

Ce travail s'adresse directement à la coordination du projet pour l'orienter dans ses choix sans avoir à manipuler le code informatique. Comme le présente le premier tableau de bilan, sur un total de 11 messages lus, 9 sont valides, 2 ont été rejetés et 6 événements distincts ont été identifiés.

Le deuxième tableau détaille les messages rejetés en indiquant leur numéro de ligne et le motif de l'erreur. Ces erreurs montrent que les deux messages rejetés contenaient des soucis de format, entraînant la perte d'informations sur des alertes importantes.

Le troisième tableau met en valeur la liste complète de tous les événements uniques après le nettoyage des doublons. Grâce à ces données, on constate que les zones les plus touchées et signalées par les habitants sont Les Cayes et Torbeck.

Le quatrième tableau permet de comparer les résultats par grand sujet d'intervention. Les résultats chiffrés montrent un total brut de 81 personnes si l'on additionne l'ensemble des messages reçus. En revanche, le décompte réel établi par événement unique donne un total de 54 personnes.

Additionner tous les messages sans distinction surestime donc considérablement les besoins de 27 personnes au total. Cette différence s'explique par le fait que de nombreux citoyens envoient plusieurs appels pour signaler une seule et même situation.

Les sujets les plus urgents à traiter d'après les tableaux concernent principalement le manque d'eau et les routes bloquées. Cette situation représente un risque réel de laisser des personnes vulnérables sans aide ciblée sur place.

Une limite importante de cette analyse réside dans le fait qu'elle dépend entièrement de la fiabilité des messages reçus. De plus, le système ne permet pas de couvrir les zones qui n'ont pas du tout de réseau téléphonique.

Pour améliorer l'efficacité des secours, nous recommandons de prioriser les actions d'urgence à Torbeck et Les Cayes. Il est également conseillé de mieux sensibiliser les citoyens sur la façon d'envoyer leurs messages pour éviter les erreurs à l'avenir.

Les décisions d'aide doivent impérativement s'appuyer sur les événements uniques pour une meilleure répartition des ressources. Un suivi rigoureux de la qualité des données évitera de mauvaises estimations lors des prochaines crises.

L'examen attentif des tableaux confirme que le tri et le nettoyage des messages sont indispensables avant toute action humanitaire. En conclusion, pour orienter utilement la décision du projet, il faut rappeler que toutes ces données analysées sont des données synthétiques.

Les tableaux

Tableau 1 : Bilan des contrôles

Élément	Valeur
Nombre de SMS lus	11
Nombre de SMS valides	9
Nombre de SMS rejetés	2
Nombre d'événements distincts	6

Tableau 2 : SMS rejetés

Ligne	Motif du rejet
6	Le nombre doit contenir uniquement des chiffres
8	4 éléments au lieu de 5

Tableau 3 : Evènements dédoublonnés

Lieu	Thème	Statut	Nombre	Premier SMS	Nombre de SMS reçus
Torbeck	EAU	URGENT	12	2026-09-03 07:12:00	3
Chantal	ROUTE	BLOQUEE	3	2026-09-03 07:31:00	2
Maniche	ABRI	URGENT	8	2026-09-03 08:02:00	1
Camp-Perrin	NOURRITURE	MOYEN	5	2026-09-03 08:15:00	1
Les Cayes	EAU	URGENT	20	2026-09-03 09:05:00	1
Port-Salut	EAU	MOYEN	6		

Tableau 4 : Tableau comparatif par thème

Thème	Personnes_si_on_additionne_les_sms	Personnes_par_évènement
ABRI	8	8
EAU	62	38
NOURRITURE	5	5
ROUTE	6	3

8. Éléments à remettre

Date limite de remise : mercredi 28 octobre 2026

- Le tableau comparatif par thème.
- Le code commenté qui génère le tableau (recopié ou en capture d'écran) avec sa sortie.
- Le rapport (au moins 20 lignes + les 4 tableaux) adressé au coordinateur ou à la coordinatrice du projet.
- Le notebook (ou le fichier .py) exécuté du début à la fin.

9. Vérification avant remise

☐ Mon tableau contient tous les thèmes et les deux colonnes de résultats.

☐ J'ai utilisé pd.DataFrame et groupby, et chaque ligne de mon code est commentée.

☐ J'ai joint mon code (recopié ou en capture d'écran) avec sa sortie.

☐ Mon rapport contient au moins 20 lignes de texte.

☐ Mon rapport intègre les 4 tableaux demandés, chacun avec un titre.

☐ Mon rapport présente des résultats chiffrés avec leurs unités et des recommandations fondées sur les constats.

☐ J'ai exécuté tout le code dans l'ordre avant la remise.

10. Barème

Critère	Détail	Points
Tableau	Tous les thèmes, deux colonnes correctes, valeurs exactes	10
Code	Utilisation de pd.DataFrame et groupby, résultat correct, chaque ligne commentée	30
Rapport	Au moins 20 lignes, 4 tableaux titrés et commentés, constats chiffrés, limites, recommandations fondées sur les résultats	60
Total		**100**